Testing Performance of Monte Carlo Tree Search and Greedy AI Agents in Uno Card Game

Stewart Almeida - CS5100 - 4/12/2026

Section I : Introduction

The implementation and use of AI agents to test their performance in the game world is nothing new - for decades now, various agents serving as players have been used on games ranging from video games, board games, and classic competitive games like Chess and Go where the former has long served as the standard “benchmark” test for various algorithms (van den Herik, 2010). The Monte Carlo Tree Search (MCTS) technique is one such search method that has seen an extensive use case for solving some of the toughest problems when it comes to successfully implementing AI when it comes to games (Chaslot et al., 2008)(Browne et al., 2012)(Palma and Lanzi, 2018). Indeed, one of the general goals is to be able to develop and test an AI agent to outperform a human player, a longstanding objective to test the effectiveness and capability of a particular algorithm. MCTS has become a particularly interesting algorithm with a great sense of focus given to its numerous variations - having solved the “challenging task of computer *Go*”, MCTS has seen extensive use as scientists and mathematicians try to extend and refine the capabilities of this search algorithm (Browne et al., 2012).

Card game implementation of AI algorithms is quite extensive. In 2017, AI Liberatus, developed by Brown and Sandholm (a successor to Claudico, an earlier developed AI agent) was able to best 4 top *Poker* pros in a showing of machine intelligence overcoming human performance in a classic yet complicated card game (Brown and Sandholm, 2017)(Spice 2017)(Yang et al., 2024). Palma and Lanzi compared various AI agents, ranging from those based around greedy algorithms to MCTS and Information Set Monte Carlo Tree Search (ISMCTS) in a popular italian card game *Scopone* - comparing different greedy algorithms with fine tuned MCTS and ISMCTS agents highlighted the strength of both MCTS variants, with even the ISMCTS outperforming agents utilizing a greedy approach back with advanced card game tactics developed over the centuries (Palma and Lanzi, 2018). Yang et al examined UNO through reinforcement learning by integrating Monte Carlo algorithms with a deep neural network. These are further combined with multi-stage learning during its training process,

creating a Multi-Deep Monte Carlo that achieved a 99% and 85% win rate over Random and Rules-Based agents respectively (Yang et al., 2024). The wide ranging utilization of MCTS, alongside the methodology and structure implemented by Palma and Lanzi, serve as a basis for this project.

To expand on work done implementing MCTS within the game field, we examine its capabilities in the game of UNO. UNO is a popular shedding-type card game first developed by Merle Robbins in 1971 with the basic objective is for a player to discard their hand completely to score points and work towards a win. Since its inception, numerous variations of the game have been created with different rule sets, cards available for use, and even platforms ranging from the original physical card set to video games. Uno itself serves as an interesting card game to examine - turn order does not strictly alternate between players given the effects of certain cards, the state of the field is incomplete at any given time, the number of players can range by a wide margin, and the overall state space can grow to be quite large. UNO was selected as the game of choice thanks to its adaptable rule set, ease of play, and unique attributes.

In this paper, we examine the performance of three AI agents developed to play through a series of UNO games. The first agent developed utilizes a greedy approach by selecting a random available action when able - this serves as our baseline “simple” player who only picks any legal move at random. The second agent developed uses a similar greedy approach with some slight modifications to implement an initial priority use case - random actions are still taken, but this agent will prioritize action cards over basic number cards. This agent represents a slightly more “advanced rules” based player. The third agent developed implements the MCTS algorithm - as MCTS requires knowledge of the state space from all players, this effectively treats it as our “cheating” player and serves as an upper bound on algorithm performance.

Section II : Methods

This section provides an overview of the methods implemented for this project, starting with an overview of the card game UNO, differences in rule sets used for the experiment, MCTS and agents utilized. Additional information on the standard cards and rule set for UNO can be found in the instructions sheet for the game (UNO, 2001).

UNO:

UNO is a shedding-type card game where the objective is for a player to discard all available cards in their hand before another player does. A running score is given to the player who first discarded their hand, based on the card values of cards remaining in the remaining player's hands. A standard UNO deck consists of 112 cards with the following breakdown:19 blue/red/yellow/green cards ranging from 0 to 9 (76 total), 8 Skip cards, with 2 of each color, 8 Reverse cards, with 2 of each color, 8 Draw 2 cards, with 2 of each color, 4 Wild cards, 4 Wild Draw 4 cards, and 4 blank cards. Each player initially draws a card - whoever draws the highest point value card serves as the dealer. Each player is dealt 7 cards from the deck. The deck is placed between the players and an initial top card is drawn. The initial state of the game depends on the card drawn.

Play initially begins to the left of the dealer. Players must try to play a legal card by matching the color, value or word of the top card on the discard pile with a card from their hand. If unable to play a legal card, a player must then draw. If the drawn card can be played, the drawer may immediately play that card. Otherwise, play advances to the next player after a card draw. When a player is about to play their next to last card, they must verbally yell out "UNO" before their card touches the discard pile. If they are caught by another player before they yell out "UNO" and their card has touched the discard pile, they must draw 4 cards. If they are not caught and play has advanced to the next player, there is no penalty issued even if caught immediately afterwards. Once a player has played their last card, the round ends. The player

with no cards left is given a score based on the card values of the remaining cards in the other player's hands.

The first player to reach 500 points is the winner. A score is given to the first player that discarded their hand, based on the card values of the remaining cards in the other player's hands. Card values are as follows: Number cards (0-9) - Face value of card, Draw 2 - 20 points, Skip - 20 points, Reverse - 20 points, Wild - 50 points and Wild Draw 4 - 50 points

A number of differences were implemented in this experiment to either reduce complexity due to time constraints or simplify some of the processes involved. First, the traditional scoring system is not used - whichever player is first able to discard their hand is effectively the winner of the match as a whole. A running score total is not kept, and each win serves as a distinct match result rather than a cumulation of different match states. This experiment focuses on strict two player gameplay - most action cards, in this case, serve as skips against their opponent when played in addition to any other effects the card may have that differ from this nature. The 4 blank cards included in a typical deck are also not utilized - these cards can serve a wide range of purposes from simple blanks to complete custom action cards, and were not factored into this project. Additionally, color selection when playing Wild cards is currently random for potential optimal/sub-optimal plays to be made. Rules regarding the first card drawn to the field at the start of the game are not included, with play beginning immediately with no immediate consequences. Agents do also not have to handle the action of "yelling" UNO when playing their second to last card - we are assuming that agents will always correctly say UNO when playing their second to last card, and the associated penalty for not doing so correctly is not factored in. Lastly, the action of passing turns even during a turn with a legal action available are not included - every turn will have a direct action/effect on the game state, be it from playing a card or drawing a card from the deck.

MCTS:

MCTS is a “best-first” search technique that iteratively builds out a search tree by taking random actions/samples from the most current node, then building out the tree from those results (Chaslot et al., 2008)(Browne et al., 2012)(James et al., 2017)(Coulom, 2006). A node represents a single game state within the tree, with the tree built out in four distinct phases as shown in Fig.1 below.

For each iteration of the process, a tree policy is used to select the next action based on a balance between exploration and exploitation - while the goal may be to select the action with the best results gathered so far through exploitation, exploration must also be considered for actions that may eventually lead to more favorable results. This policy, typically the Upper Confidence Bound for Trees (UCB1 for trees or UCT) is used to descend from a child node down the tree towards a leaf node or a not fully expanded node that maxes out the UCT value (Palma and Lanzi, 2018)(Auer et al., 2002). UCT values are computed via the following equation

$$\frac{P(v')}{Q(v')} + c\sqrt{2\frac{ln(Q(v))}{Q(v')}}$$

where v and v' represent the parent and child nodes respectively, $P(v')$ represents the total rewards of playouts through node v', $Q(v)$ and $Q(v')$ represent the number of times nodes v and v' are visited respectively, and c represents the constant used to represent the exploration weight.

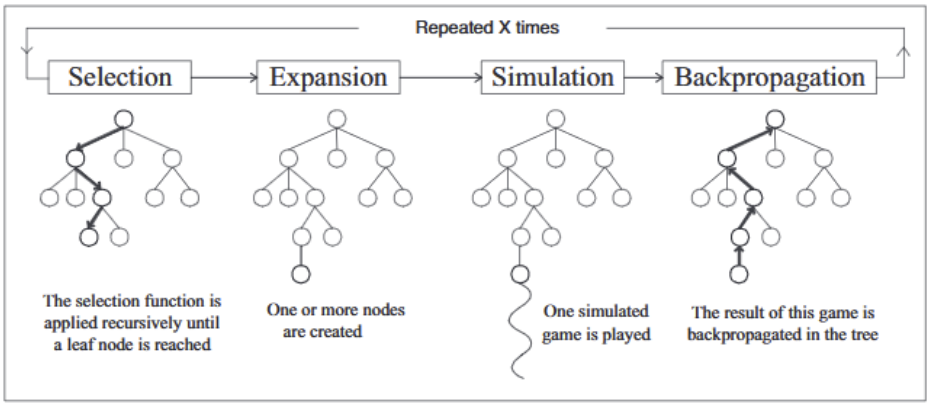

Fig 1. The four phases of the MCTS process (Chaslot et al., 2008)

If that node selected does not represent a terminal state, child nodes are added on to expand the tree one node at a time. Simulations are run from the current tree position according to a rollout policy - whatever the outcome is, that value is backpropagated back up the tree through the nodes visited, with the nodes' average scores and visit counts being updated. This cyclic process is repeated until some halting criteria is met, with the best action taken based, in this case, on the child node with the most visits.

Agents:

Three agents were developed for this project that serve as various different "players" in a series of UNO games, implementing different strategies against their opponent. The first agent developed is a greedy Random agent (RA). This RA follows a very simple strategy - determine all possible legal moves from the current state based on the cards in their hand, and pick a move at random. The RA serves as the lower bound for performance, only selecting a random legal move without consideration for more favorable actions.

The second agent developed is a priority based agent (PA). The PA is similar to the RA in that it selects a legal action at random, but with one major modification. This agent will

prioritize using the special action cards first before resorting to playing basic number cards. The main motivation behind this particular agent is to represent a slightly more “advanced” player that will, in a two player setting, attempt to utilize cards that disrupt their opponent’s turn order first to more quickly reduce the number of available cards in their hand. As most action cards save for the pure Wild card all skip the opponents turn, the PA will attempt to shed their hand before their opponent can even play a card.

The third and final agent developed is the MCTS agent (MA). The MA serves as an upper bound of algorithmic performance - as the MCTS algorithm requires knowledge of the states of both players in this case, the MA effectively serves as a “cheating” player that has greater knowledge of the state of the game than what is normally available.

The agents were paired up to create three distinct match ups - RA vs PA, RA vs. MA, and PA vs. MA. Hyperparatemer tuning values for the MA are described in Section III. The number of games played between each agent was set to 100, providing a sizable yet computationally budget friendly number of games to test against each matchup. Additionally, each game swapped which agent played first to help reduce the bias on initial turn order based on the acting agent (Palma and Lanzi, 2018)(Gelly and Wang, 2006).

Section III: Results

Determining the optimal value for hyperprameters (exploration factor and number of iterations) will significantly affect the performance of the MA - the number of iterations will affect the overall computing power needed to actually run the MA against another opponent, while the exploration factor balances the exploration/exploitation strategy used in the UCT. As these values are not fixed based on the problem set, different games, different versions of the same game, and any modifications specific to one particular game will all potentially call for varying hyperparatemer tuning. To begin the process of tuning the MCTS appropriately, a heat map was

generated by comparing different iteration values and weights for the MCTS against an RA as an opponent. Both agents played 100 games. See Fig. 2 below for the heatmap generated.

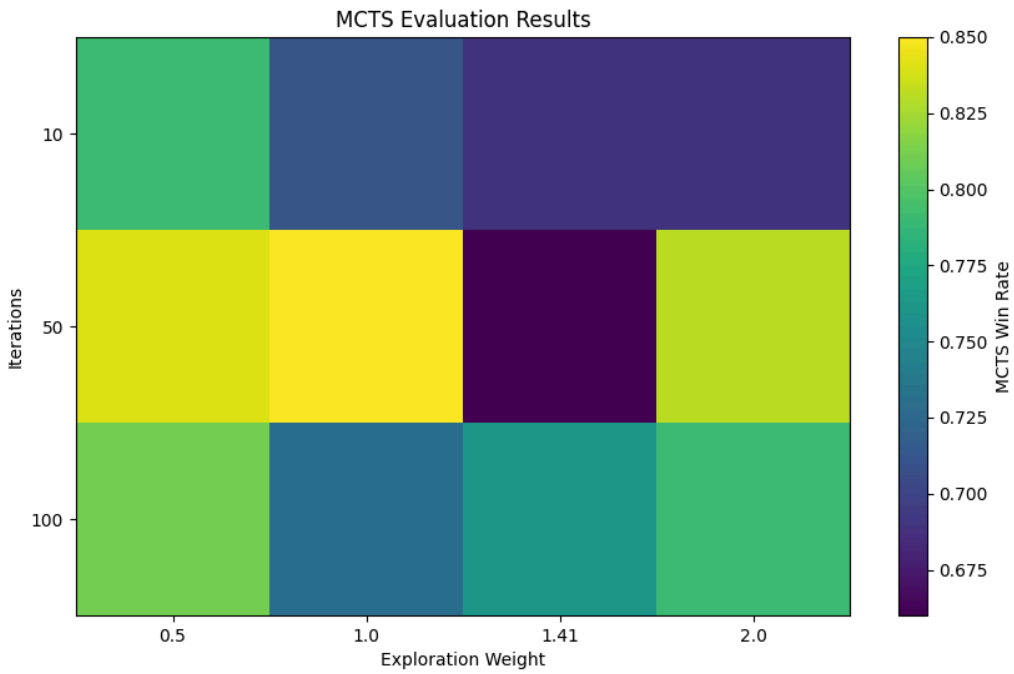

Fig 2. Heatmap comparing MCTS win rates with varying iterations and exploration weights

Increasing the number of iterations generally increased the overall win rate of the MA against the RA. Testing different values of the exploration weight highlighted how the balance between exploration and exploitation can lead to varying results - as a result, the number of iterations was set to 50 with an exploration weight constant of 1. This struck a fine balance between the computational budget required to run the MA agent and optimizing the exploration weight constant, where both lead to the highest overall winrate. While a higher number of iterations might lead one to expect better performance, the computational workload increase, combined with lower win rate values, pushed the number of ideal iterations down.

The overall win rate results between the three distinct matchups are displayed in Fig 3. Between the RA and PA, we can note that the PA out performed the RA with a win rate of 61% to 39% respectively out of 100 games played. These results were expected at some level, with the difference in win rates highlighting the value of implementing a basic rule set/policy when it comes to playing legal moves. While both the RA and PA both execute their moves randomly, the PA, with its focus on playing action cards before any basic number card, had much more favorable success in overcoming the RA. Given the nature of two player Uno and the value or effect these action cards had, placing a small focus on effectively skipping the opponent RA's turn more often proved a solid win.

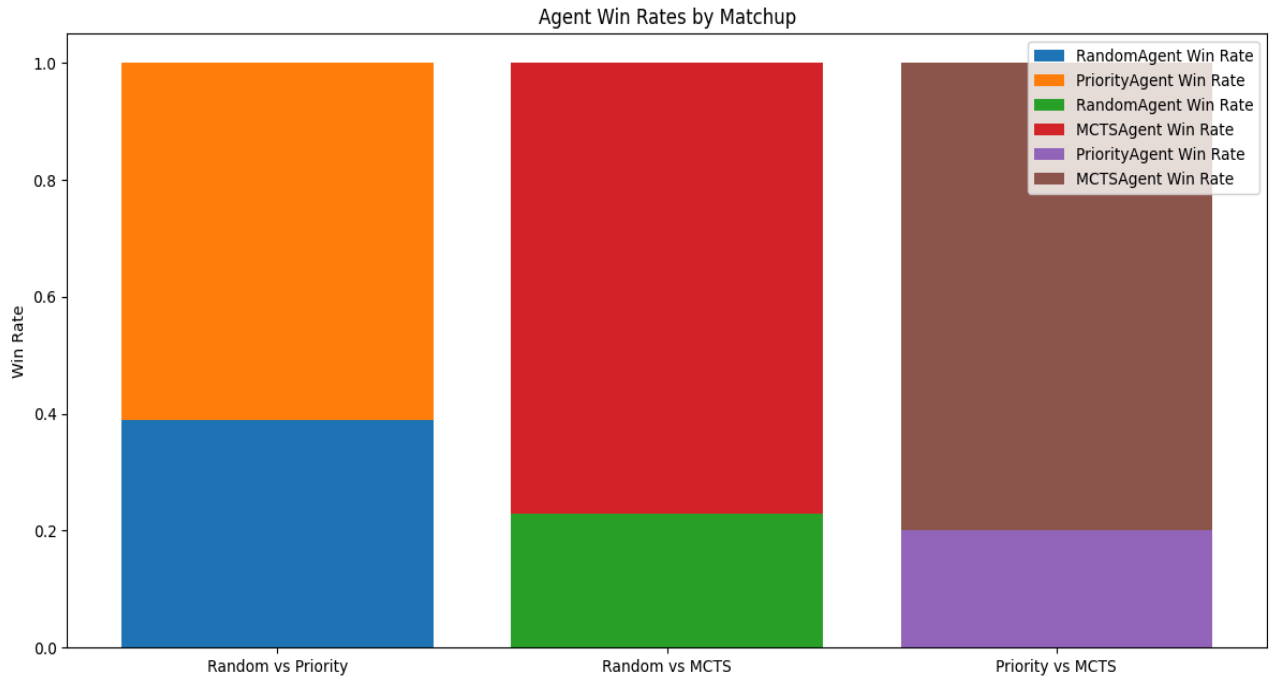

Fig 3. Win Rate Matchup Comparisons

Looking at the matchup between the RA and MA, we see that the MA held strong with a win rate of 77% compared to the RA's 23%. Again, these results were also to be expected but

the actual differential between the two showcases the strength of an informed search tree compared to pure uniformity randomness. Interestingly, the matchup between the PA and MA actually yielded even more favorable results to the MA - in this case, the MA held a win rate of 80% compared to the PA's 20%. Overall, the MA outperformed the RA and PA in both matchups decidedly, though the differential between the two MCTS matchups calls forth a number of questions despite the expected outcome. The RA selects its action uniformly at random, ignoring any consequences this action may have for the future. The PA also selects its action at random, though it first will prioritize a random action concerning an action card. While the MA also deploys some randomness within its rollout (simulation) phase, the actual decision making the agent makes is guided by repeated sampling and a statistical evaluation of the outcomes encountered. Thanks to these numerous simulations, the MCTS is able to estimate the long term value of the actions taken, biasing towards those to lead to higher win rates.

Section IV: Conclusion

In this project, we tested the performance of three developed agents employing different strategies against each other through the card game UNO: one based on the Monte Carlo Tree Search, one based on a priority system that will attempt to use action cards first, and one that only randomly picks a legal action. The results indicated that out of the three agents, the RA performed the worst when paired up against the PA and MA. The MA performed the best, attaining wins rates of 77% and 80% against its RA and PA opponents respectively. Between the two MCTS matchups, the MA performed better against the PA than the RA by 3%.

This project holds many potential changes and upgrades that may provide additional insight to strengthen or broaden the results gathered. Many of the restraints employed were necessitated out of computational capability or time constraints. As discussed previously, the traditional scoring system used in UNO was not employed to treat each game played as distinct rather than a cumulation of different games. Employing the actual scoring system, while not only

increasing the total number of “games” played, may offer insight on the trends between individual matches within a larger “match” set. Additionally, there could be further modifications to the other rules employed, namely the randomness involved with Wild cards. In a standard game, a player would opt to use a Wild card on a color that favors them more rather than at random. Additional guard rails would need to be in place to prevent Wild cards to be played when the current state already favors the player, but it could skew more favorable results for the PA. Similarly, the MA could also employ a more heuristic approach in its rollout rather than the randomness it currently uses. While it is true that, overtime, the MA will be biased towards actions that lead to more wins, being smarter about which actions are used in the interim during the various simulations may lead to even greater results for the MA. Iteration values and even depth of the tree were locked at maximum values for the experiment - compared to the values used when working through *Scopone*, the values employed here are a few orders of magnitude different, which can heavily affect the tuning of the MA (Palma and Lanzi, 2018). Lastly, the use of an ISMTCS agent (IA) would offer a more realistic or comparable look at a “normal” player compared to the MA, given that in a normal UNO game, a player only has an incomplete information space to actively base their decisions and actions on. Overall, the work done here would serve as a respectable foothold for future endeavors in experimenting with AI performance and the card game UNO.

Works Cited

H. J. van den Herik, "The Drosophila revisited", Int. Comput. Games Assoc. J., vol. 33, no. 2, pp. 65-66, 2010.

Chaslot, G., Bakkes, S., Szita, I., & Spronck, P. (2008). Monte-Carlo Tree Search: A New Framework for Game AI. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, *4*(1), 216-217. https://doi.org/10.1609/aiide.v4i1.18700

Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., & Colton, S. (2012). A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games*, *4*(1), 1–43. https://doi.org/10.1109/tciaig.2012.2186810

Palma, S. D., & Lanzi, P. L. (2018). Traditional Wisdom and Monte Carlo Tree Search Face-to-Face in the Card Game Scopone. *IEEE Transactions on Games*, *10*(3), 317–332. https://doi.org/10.1109/tg.2018.2834618

UNO [Card game]. (2001). Mattel. https://service.mattel.com/instruction_sheets/42001pr.pdf

James, S., Konidaris, G., & Rosman, B. (2017). An Analysis of Monte Carlo Tree Search. *Proceedings of the AAAI Conference on Artificial Intelligence*, *31*(1). https://doi.org/10.1609/aaai.v31i1.11028

Coulom, R. (2006). Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search. *Hal.science*. https://inria.hal.science/inria-00116992

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, *47*(2/3), 235–256. https://doi.org/10.1023/a:1013689704352

Brown, N., & Sandholm, T. (2017). *Safe and Nested Subgame Solving for Imperfect-Information Games*. ArXiv.org. https://arxiv.org/abs/1705.02955

Spice, B. (2017). *Upping the Ante: Top Poker Pros Face Off vs. Artificial Intelligence | Carnegie Mellon University Computer Science Department*. (2017). Cmu.edu. https://www.csd.cs.cmu.edu/news/upping-the-ante-top-poker-pros-face-off-vs-artificial-intelligence

Yang, X., Liu, X., & Lin, W. (2024). Multi-DMC: Deep Monte-Carlo with Multi-Stage Learning in the Card Game UNO. *2021 IEEE Conference on Games (CoG)*, 1–8. https://doi.org/10.1109/cog60054.2024.10645662

Gelly, S., & Wang, Y. (2006). *Exploration exploitation in Go: UCT for Monte-Carlo Go*. HAL Archives Ouvertes. https://hal.science/hal-00115330